

Master Practice Workbook & Portfolio Projects (Phases 1 & 2)

This document represents the consolidated, production-grade practice curriculum and portfolio projects for Phase 1 and Phase 2 of the Elite AI/ML roadmap. This material spans core Python software design, exception handling frameworks, vectorized matrix manipulation, advanced relational database querying, and complete end-to-end data pipelines.

1. Phase 1 Practice Problems: Core Python Frameworks

Problem 1.1: Custom Object-Oriented Inventory Management System (OOP & Exceptions)

Scenario: Build a modular backend core to track inventory stock for an e-commerce infrastructure array. Your code must follow strict encapsulation principles.

1. Create a base class `Product` with encapsulated attributes: `_product_id` (protected), `__name` (private), and `__price` (private). Expose safe getter methods.
2. Derive a child class `DigitalProduct` that inherits from `Product` but includes a unique property for `download_link`. Override a method named `get_product_type()` to demonstrate polymorphism.
3. Implement a custom user-defined exception named `InsufficientStockError`. Raise this exception if a transaction attempts to reduce inventory levels below zero. Handle this failure cleanly inside a `try-except-finally` architecture block.

Problem 1.2: Automated Regex Log Parser & Stream Writer (File Handling)

Scenario: You are handed a raw text file containing thousands of server logs. You need to parse anomalies without loading the entire text structure into RAM simultaneously.

1. Utilize a context manager (with statement) to stream an input text file line by line.
2. Parse each line to capture string lines that match a specific error signature framework (e.g., lines containing "ERROR [500]" or "CRITICAL").
3. Write those captured lines safely into an isolated file named `anomaly_report.txt`. Ensure the append mode is handled correctly so that concurrent executions don't wipe previous

data rows.

Problem 1.3: Dynamic Execution Metrics Wrapper (*args, **kwargs, Lambda)

Scenario: Write a utility performance tracking wrapper function that can ingest any arbitrary function configuration and record call details.

1. Write a function named `execution_wrapper(target_function, *args, **kwargs)`.
2. Inside the wrapper, log the total number of positional variables passed (using `len(args)`) and print the specific keywords captured inside `kwargs`.
3. Pass an inline anonymous lambda expression evaluating multi-variable polynomial equations (e.g., `lambda x, y: (x**2) + (y**3)`) through the execution wrapper to verify functional performance.

2. Month 1 Portfolio Project: Smart File Organizer & Resilient Log Parser

Objective: Combine system-level automation with regular expression data extraction to build a highly stable background utility package.

Architectural Directory Setup

```
smart-file-organizer/
├─ main.py           # Core orchestration engine execution
file
├─ file_engine/      # Package handling system OS disk
modifications
|   └─ __init__.py
|   └─ organizer.py
└─ parser_engine/    # Package running string regex streaming
algorithms
    └─ __init__.py
    └─ log_parser.py
```

Core Requirements

- **Automated Directory Cleaner:** Write a module utilizing os and shutil libraries to scan a target directory, inspect file formats, and automatically sort files into matching destination directories (e.g., .pdf -> docs/, .csv -> data/).
- **Memory-Efficient Streaming Log Parser:** Create a regex-powered engine that opens raw text logs via generators to dynamically stream log entries line-by-line. Capture all anomalies matching error signatures and write them out to a structured data summary file.
- **Exception Resilience:** Implement robust catch structures handling missing file permissions (PermissionError), disk reading interruptions, or file structural deletions without halting execution.

3. Phase 2 Practice Problems: Data Engineering, SQL, & EDA

Problem 2.1: Matrix Z-Score Normalization Pipeline (NumPy Broadcasting)

Scenario: Before feeding values into gradient descent scripts, features must be scaled to eliminate scale bias. You must implement this vectorization mathematically using NumPy without loops.

1. Generate a mock random 2D matrix of shape (100, 5) using np.random.randn(), representing 100 sample data rows across 5 distinct feature columns.
2. Compute the feature-wise mean vector (shape (5,)) and standard deviation vector (shape (5,)) across the column axis (axis=0).
3. Leverage NumPy broadcasting parameters to normalize the entire matrix down to a standard normal scale using the mathematical Z-score formula:

$$Z = (X - \text{Mean}) / \text{Standard_Deviation}$$
Verify that the output matrix mean converges perfectly to 0.

Problem 2.2: Tabular Churn Diagnostic Processing Engine (Pandas Aggregation)

Scenario: You are handed a raw customer transaction data frame containing massive data gaps and null parameters. You must structure an ETL aggregation loop.

1. Locate all missing data cells inside a DataFrame column named Account_Balance. Impute those missing data blocks with the median value calculated from the non-null rows of that column.
2. Utilize the groupby() framework to segment rows based on two distinct features:

Geographic_Region and Subscription_Tier.

3. Apply a compound multi-aggregation calculation to pull three metrics per group: the mean of Monthly_Charges, the maximum of Customer_Tenure, and the absolute sum of the Churn_Event binary flag column.

Problem 2.3: Multi-Tiered Transaction Fraud Audit Query (Advanced SQL)

Scenario: Write an analytical SQL query to extract fraudulent activities from an enterprise banking database consisting of a users table and a transactions table.

1. Join the users table with the transactions table using a LEFT JOIN mapping on the user_id primary validation key.
2. Filter out rows using a WHERE clause to only isolate transactions executed after January 1, 2026, where the status flag is not set to 'Cancelled'.
3. Implement a nested HAVING subquery parameter that aggregates data by user_id and restricts the final selection output rows to only return users whose total combined transaction value (SUM(amount)) exceeds the overall average transaction size across the entire corporate history table.

4. Month 2 Data Pipeline Project: IPL Analytics & Data Cleaning Engine

Objective: Process raw, highly heterogeneous cricket sports data tables, performing robust missing data fixes and computing complex aggregated metrics without slow Python loops.

Core Pipeline Objectives

1. **Ingestion & Structuring:** Parse a dirty dataset of historical deliveries. Isolate columns, flag complete structural data omissions, and impute numerical feature columns with localized statistical medians using Pandas.
2. **Vectorized Feature Extractor:** Utilize vectorization paradigms inside NumPy to run massive calculations across millions of array indices without explicit loop iterations. Compute strike-rates, bowling efficiencies, and weighted impact factors natively.
3. **Multi-Dimensional Aggregation Matrix:** Deploy Pandas groupby() mechanics to build an analytical feature space mapping performance against discrete compound categorical indices (e.g., [Player_Name, Match_Venue, Innings_Phase]), returning high-impact summary profiles.

```
# Expected End-to-End Execution Pipeline Workflow:
```

```
# 1. Raw Data Extraction -> 2. Pandas Null-Imputation -> 3. NumPy  
Feature Vectorization -> 4. GroupBy Analysis Mapping
```